

การประมวลผลและการสร้าง Expression Tree

ระบบประมวลผลนิพจน์คณิตศาสตร์ด้วยโครงสร้างข้อมูล Binary Tree
พัฒนาโดยภาษา Java

Algorithm A · Stack / Postfix

Algorithm B · Recursion / Prefix

สมาชิกกลุ่ม

1

นายเนติธร กาฬภักดี

2

นายนิธิต สุขอยู่

3

นายพีรพัฒน์ ไคร์ในธรรม

4

นางสาวทิพวรรณ พิมพ์มาตร

5

นายภูรินทร์ ภาววงษ์

6

นายภูมิภัทร ศรีสุวรรณ

7

นางสาวกัญญาดา ใจบาน

PROBLEM ANALYSIS

การวิเคราะห์ปัญหา

การประมวลผลนิพจน์คณิตศาสตร์ในระบบคอมพิวเตอร์มีความซับซ้อนเนื่องจากลำดับความสำคัญของเครื่องหมาย (Operator Precedence) และการจัดกลุ่มด้วยวงเล็บ

โครงสร้างข้อมูล Expression Tree ซึ่งเป็น Binary Tree ชนิดหนึ่ง ถูกนำมาใช้เพื่อแปลงนิพจน์ให้อยู่ในรูปของโครงสร้างต้นไม้ ทำให้สามารถประมวลผล แปลงรูปแบบนิพจน์ (Infix, Prefix, Postfix) และคำนวณผลลัพธ์ได้อย่างถูกต้องแม่นยำ

INFIX

$((8+2) * (9-5)) / 2$

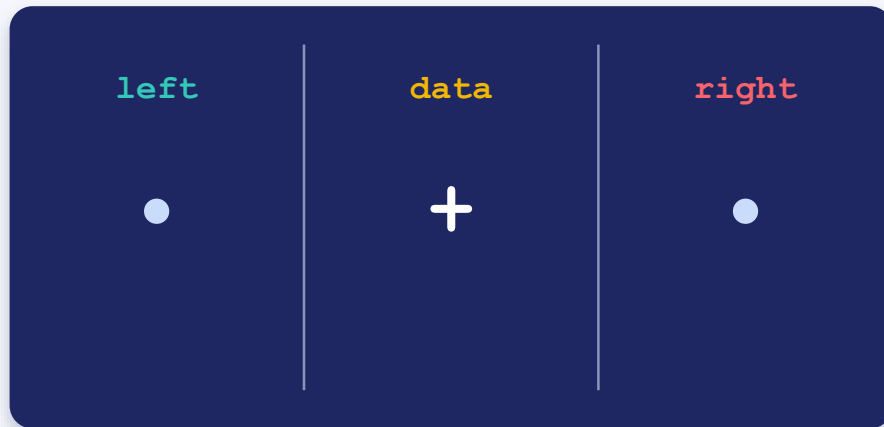
POSTFIX

8 2 + 9 5 - * 2 /

PREFIX

/ * + 8 2 - 9 5 2

การกำหนดข้อมูลที่จัดเก็บใน Node



โครงสร้าง Node หนึ่งหน่วยของ Expression Tree

D

data

จัดเก็บตัวเลข (Operand) หรือเครื่องหมายทางคณิตศาสตร์ (Operator)

L

left

ตัวชี้ไปยังโหนดลูกฝั่งซ้าย (Left Child Pointer)

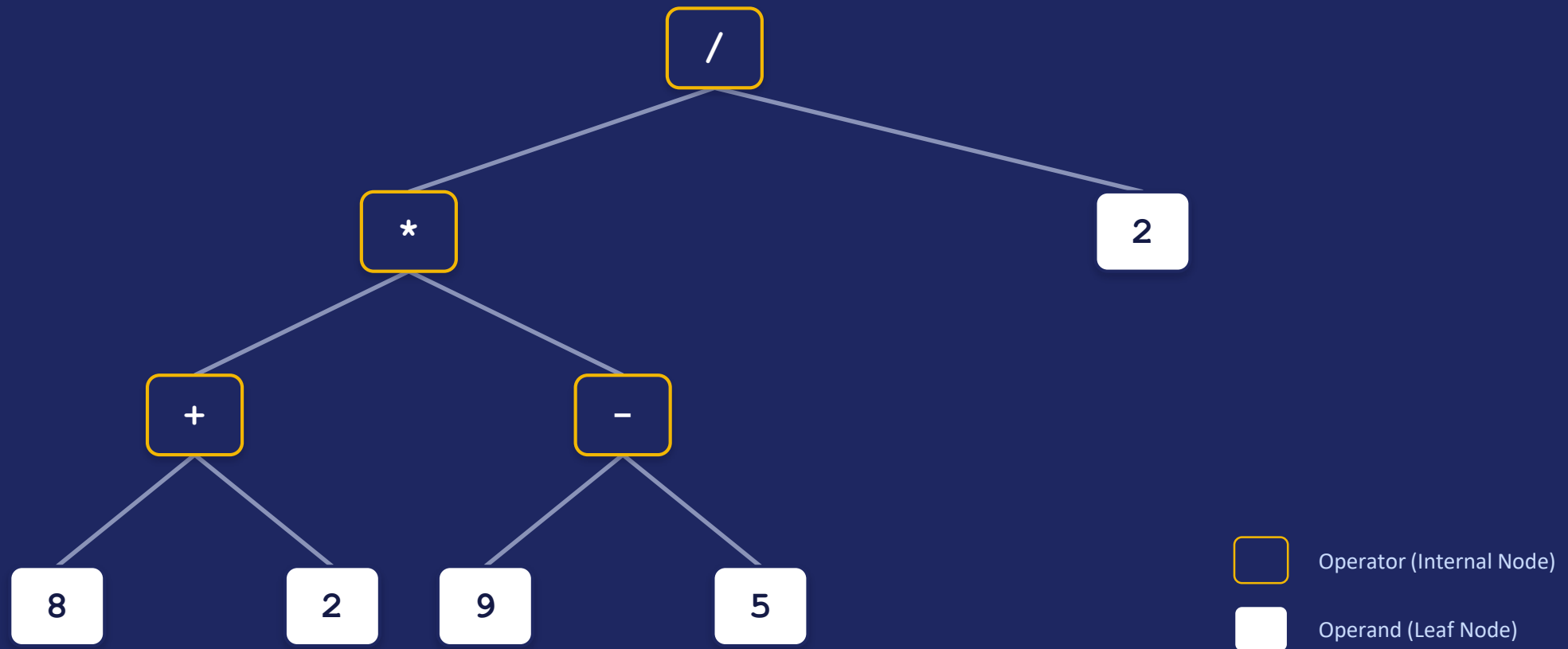
R

right

ตัวชี้ไปยังโหนดลูกฝั่งขวา (Right Child Pointer)

MANDATORY CASE 4

การวาดโครงสร้าง **Tree** จากข้อมูลตัวอย่าง
 $((8+2) * (9-5)) / 2$



การออกแบบ Algorithm A และ Algorithm B

ALGORITHM A

Postfix Build Tree using Stack

วนลูปอ่าน Token จากซ้ายไปขวา หากพบ Operand ให้สร้าง Node แล้ว Push ลง Stack หากพบ Operator ให้ Pop ออกมา 2 โหนดเพื่อเชื่อมเป็นลูกขวาและลูกซ้ายตามลำดับ แล้ว Push โหนด Operator กลับลง Stack

ลักษณะ: *Iterative* · ใช้ *Stack*

ALGORITHM B

Prefix Build Tree using Recursion

อ่าน Token จากซ้ายไปขวา ใช้การเรียกตัวเอง (Recursion) โดยสร้าง Root Node ก่อน แล้วส่งต่อไปสร้าง Subtree ฝั่งซ้ายและฝั่งขวาตามลำดับ

ลักษณะ: *Recursive* · ใช้ *Call Stack* ของระบบ

BUILD_TREE_POSTFIX (ใช้ Stack)

```
Algorithm BUILD_TREE_POSTFIX(tokens):  
    stack = NEW Stack()  
    FOR EACH token IN tokens DO  
        node = NEW Node(token)  
        IF node.isOperator() THEN  
            node.right = stack.pop()  
            node.left  = stack.pop()  
        END IF  
        stack.push(node)  
    END FOR  
    RETURN stack.pop()
```

1

Operand → สร้าง Node แล้ว Push ลง Stack ทันที

2

Operator → Pop 2 โหนดล่าสุด (right ก่อน, left ที่หลัง)

3

จบการวนลูป Stack จะเหลือ Root Node เพียงตัวเดียว

BUILD_TREE_PREFIX (ใช้ Recursion)

```
Global index = 0
```

```
Algorithm BUILD_TREE_PREFIX(tokens):  
  IF index >= tokens.length THEN RETURN null  
  token = tokens[index]  
  index = index + 1  
  node = NEW Node(token)  
  IF node.isOperator() THEN  
    node.left = BUILD_TREE_PREFIX(tokens)  
    node.right = BUILD_TREE_PREFIX(tokens)  
  END IF  
  RETURN node
```

1

ใช้ตัวแปร index แบบ Global เพื่อติดตามตำแหน่ง Token

2

สร้าง Root Node ก่อน แล้วเรียกซ้ำสร้าง Subtree ซ้าย-ขวา

3

ความลึกของ Recursion เท่ากับความสูงของ Tree

Algorithm A: Postfix 8 2 + 9 5 - * 2 /

STEP	ACTION	STACK STATE / RESULT
1	Read 8	Stack: [8]
2	Read 2	Stack: [8, 2]
3	Read +	Pop 2 (R), Pop 8 (L) → (+) → Stack: [(+)]
4	Read 9	Stack: [(+), 9]
5	Read 5	Stack: [(+), 9, 5]
6	Read -	Pop 5 (R), Pop 9 (L) → (-) → Stack: [(+), (-)]
7	Read *	Pop (-) (R), Pop (+) (L) → (*) → Stack: [(*)]
8	Read 2	Stack: [(*), 2]
9	Read /	Pop 2 (R), Pop (*) (L) → (/) → Stack: [(/)]
10	Done	Pop (/) คำนวณเป็น Root Node สมบูรณ์

CORRECTNESS

การอธิบาย Correctness และ Invariants

LOOP INVARIANT — Algorithm A

ทุกขั้นตอนที่เสร็จการทำงาน ทุกโหนดที่อยู่ใน Stack จะต้องเป็น Root Node ของ Subtree ที่ถูกต้องตามลำดับนิพจน์ Postfix เสมอ

TREE STRUCTURE INVARIANTS

1 Leaf Node

ทุกโหนดที่เป็น Leaf ต้องเป็น Operand (ตัวเลข) เสมอ

2 Internal Node

ทุกโหนดที่เป็น Internal ต้องเป็น Operator และมี Subtree ลูกครบทั้งสองฝั่ง (Full Binary Tree)

การวิเคราะห์ Time Complexity

Build Tree

Algorithm A & B

$O(N)$

N คือจำนวน Token ทั้งหมดในนิพจน์

Evaluate

คำนวณผลลัพธ์

$O(N)$

ต้อง Visit ทุกโหนดเพื่อคำนวณผล

Traversal

In / Pre / Post Order

$O(N)$

ต้องผ่านทุกโหนด โหนดละ 1 ครั้ง

การวิเคราะห์ Space Complexity

AUXILIARY MEMORY SPACE

$O(N)$

สำหรับการสร้างและเก็บ N Nodes ในหน่วยความจำ Heap

CALL STACK SPACE

$O(H)$

H คือความสูงของ Tree (Height)

Best Case (Balanced Tree)

$O(\log N)$

Worst Case (Skewed Tree)

$O(N)$

การพัฒนาโปรแกรม Java

1

Node.java

นิยาม Class และสถานะ isOperator()

2

ExpressionTree.java

เก็บวิธีสร้างและประมวลผล Tree (buildFromPostfix, evaluate, printInfix ฯลฯ)

3

Main.java

จุดเริ่มต้นการทำงานและทดสอบคำนวณผลลัพธ์

TESTING

จัดทำ Test Cases (8 กรณีทดสอบ)

TYPE	POSTFIX INPUT	EXPECTED OUTPUT
Single Operand	5	5.0
Simple Addition	3 4 +	7.0
Operator Precedence	3 4 2 * +	11.0
Mandatory Case 4	8 2 + 9 5 - * 2 /	20.0
Subtraction & Negative Result	0 5 - 3 *	-15.0
Division with Decimal	7 2 /	3.5
Long Expression	1 2 + 3 4 + *	21.0
Large Numbers	100 200 + 50 *	15000.0

Edge Cases (3 กรณีขอบเขต)

1

Division by Zero

PROBLEM: ตัวหารมีค่าเป็น 0 (5 0 /)

HANDLING STRATEGY

คืนค่า Infinity ตามมาตรฐานประเภทข้อมูล Double

2

Single Node Tree

PROBLEM: นิพจน์มีเฉพาะเลขตัวเดียว (9)

HANDLING STRATEGY

ระบบประมวลผลคืนค่าตัวเลขทันทีโดยไม่ต้องเข้า Subtree

3

Highly Skewed Tree

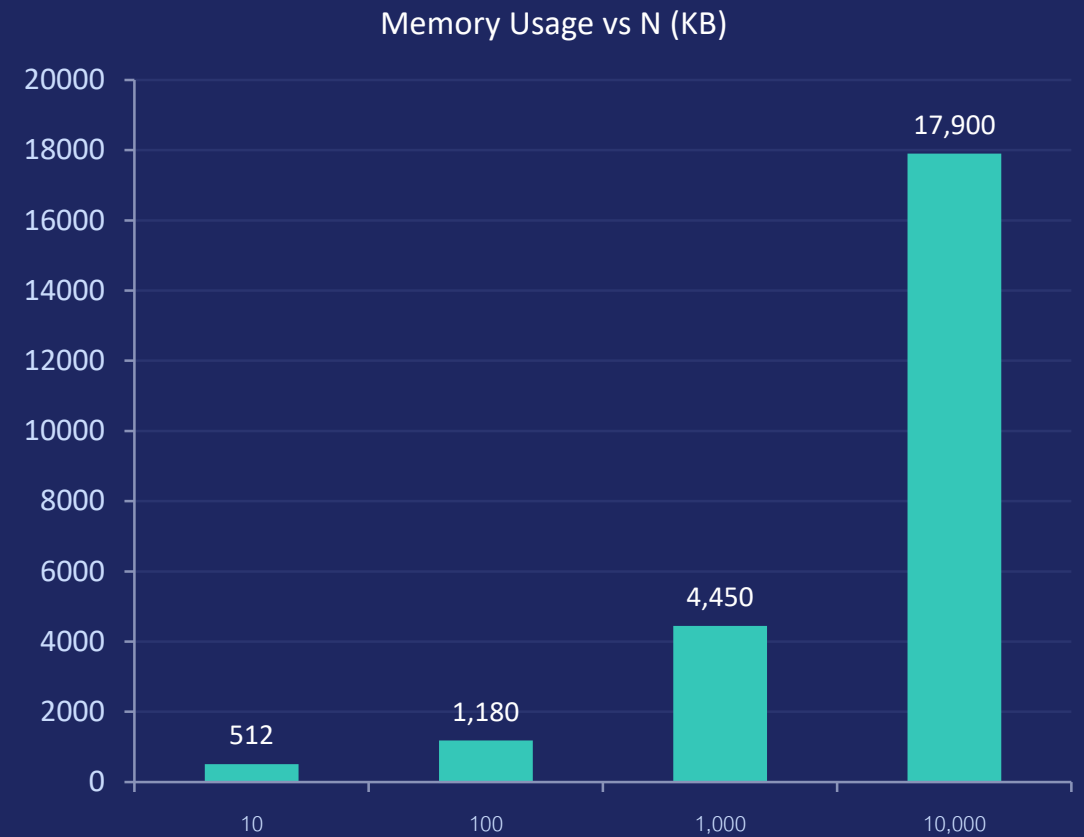
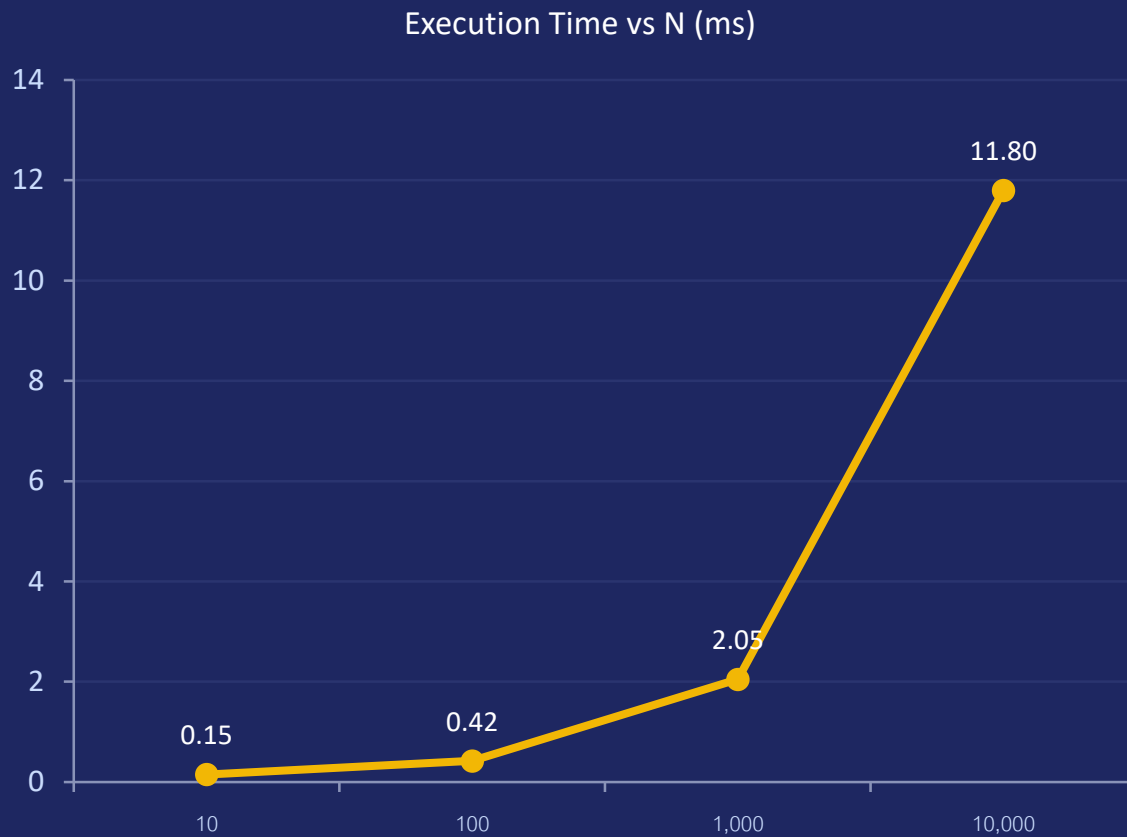
PROBLEM: นิพจน์เอียงไปฝั่งเดียวอย่างยาว

HANDLING STRATEGY

ประมวลผลผ่าน Iterative Stack เพื่อป้องกันการเกิด StackOverflowError

EXPERIMENTS

การทดลองด้วยข้อมูลหลายขนาด



CONCLUSION

สรุปผลและข้อเสนอแนะ

จากการทดลอง เวลาในการประมวลผลเพิ่มขึ้นแบบเชิงเส้นตามจำนวน Token (N) ซึ่งสอดคล้องกับ Time Complexity เชิงทฤษฎีระดับ $O(N)$ อย่างแน่นอน

Algorithm ไດเหมาะสมกว่าในสถานการณ์ใด

ALGORITHM A

เหมาะสำหรับ **Production System**

ใช้ Iterative Loop ทำให้ไม่เกิดความเสี่ยงเรื่อง StackOverflowError แม้ว่า Tree จะมีความลึกมาก

ALGORITHM B

เหมาะสำหรับการศึกษา

เหมาะสำหรับนิพจน์ที่มีขนาดเล็กและแน่นอน เนื่องจากเขียนได้สั้น กระชับ และทำความเข้าใจง่ายกว่า